

Junction trees and belief propagation

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	The junction tree of asia	2
4	Calibration and all marginals	3
5	Belief propagation on a tree	5
6	Loopy belief propagation on asia	5
6.1	Slow updates are not convergence	7
6.2	Unknown feasibility on a loop	7
7	Timing: all marginals by junction tree versus variable elimination	8
8	Summary	9
9	References	9

1 Overview

Variable elimination answers one query per run. When every marginal of a network is wanted (an interactive “what does the evidence do to everything” display, or a sensitivity sweep), two further backends work on the same FactorGraph:

- `JunctionTree()` builds a clique tree of the interaction graph with [CliqueTrees.jl](#) (`cliquetree(graph; alg=MF(), snd=Maximal())`), calibrates it by Shafer-Shenoy ([Shafer and Shenoy 1990](#)) message passing, and reads every marginal off the calibrated beliefs; the cost is two sweeps over the tree, whatever the number of variables queried.
- `BeliefPropagation()` runs sum-product message passing on the factor graph itself. On a tree-structured factor graph (a polytree network) it is exact; on a graph with loops it is the loopy approximation, with convergence diagnostics.

2 Setup

```
using BayesianNetworkInference
using FiniteKernels
using BayesianNetworks
using BayesianNetworkFormats: fixture_path
using Random
```

3 The junction tree of asia

The asia network is read from the BIF fixture and compiled to a factor graph, one factor per mechanism.

```
asia = read_bayesnet(fixture_path("bif/asia.bif"))
fg = compile(asia)
```

FactorGraph{Float64} with 8 factors over 8 variables

`build_junction_tree` triangulates the moral graph with a fill-reducing order and returns the maximal cliques of the result as a rooted tree. Each non-root clique lists the separator it shares with its parent.

```
jt = build_junction_tree(fg)
```

CompiledJunctionTree with 6 cliques (treewidth 2)

```
1: [:asia, :tub] --[:tub]--> 2
2: [:tub, :lung, :either] --[:lung, :either]--> 6
3: [:either, :xray] --[:either]--> 6
4: [:bronc, :either, :dysp] --[:bronc, :either]--> 5
5: [:smoke, :bronc, :either] --[:smoke, :either]--> 6
6: [:smoke, :lung, :either]
```

```
(n_cliques=length(jt), treewidth=jt.treewidth, roots=jt.roots)
```

```
(n_cliques = 6, treewidth = 2, roots = [6])
```

Every factor is assigned to the smallest clique containing its scope, and each variable records the smallest clique it lives in (where its marginal is read from):

```
[fg.provenance[i].variable ⇒ jt.cliques[c] for (i, c) in enumerate(jt.assignment)]
```

```

8-element Vector{Pair{Symbol, Vector{Symbol}}}:
 :asia => [:asia, :tub]
 :tub => [:asia, :tub]
 :smoke => [:smoke, :bronc, :either]
 :lung => [:smoke, :lung, :either]
 :bronc => [:smoke, :bronc, :either]
 :either => [:tub, :lung, :either]
 :xray => [:either, :xray]
 :dysp => [:bronc, :either, :dysp]

```

```
Dict{v => jt.cliques[c] for (v, c) in jt.home}
```

```

Dict{Symbol, Vector{Symbol}} with 8 entries:
 :tub    => [:asia, :tub]
 :xray   => [:either, :xray]
 :bronc  => [:bronc, :either, :dysp]
 :either => [:either, :xray]
 :smoke  => [:smoke, :bronc, :either]
 :lung   => [:tub, :lung, :either]
 :dysp   => [:bronc, :either, :dysp]
 :asia   => [:asia, :tub]

```

4 Calibration and all marginals

calibrate conditions the factors on the evidence, multiplies them into their cliques, and passes messages from the leaves to the root and back. The beliefs are then $P(\text{clique}, \text{evidence})$, so their total mass is the probability of the evidence.

```
cal = calibrate(fg; evidence=Dict(:asia => :yes, :xray => :yes))
```

```
CalibratedJunctionTree{Float64} with 6 cliques, P(evidence) = 0.001450925
```

```
[scope(b) => round(sum(b.table); digits=6) for b in cal.beliefs]
```

```

6-element Vector{Pair{Vector{Symbol}, Float64}}:
      [:tub] => 0.001451
  [:tub, :either, :lung] => 0.001451
      [:either] => 0.001451
[:bronc, :either, :dysp] => 0.001451
[:smoke, :bronc, :either] => 0.001451
[:either, :smoke, :lung] => 0.001451

```

all_marginals does this once and returns every single-variable posterior; observed variables appear as point masses. The junction-tree answers agree with variable elimination to floating-point precision.

```
ev = Dict(:asia => :yes, :xray => :yes)
jt_marginals = all_marginals(fg; evidence=ev)
ve_marginals = all_marginals(fg; evidence=ev, backend=VariableElimination())
sort([v => round(m.table; digits=4) for (v, m) in jt_marginals]; by=first)
```

```
8-element Vector{Pair{Symbol, Vector{Float64}}}:

```

```
 :asia => [1.0, 0.0]
 :bronc => [0.4911, 0.5089]
 :dysp => [0.6811, 0.3189]
 :either => [0.6906, 0.3094]
 :lung => [0.3715, 0.6285]
 :smoke => [0.637, 0.363]
 :tub => [0.3377, 0.6623]
 :xray => [1.0, 0.0]
```

```
maximum(maximum(abs.(jt_marginals[v].table .- ve_marginals[v].table)) for v in
variables(fg))
```

```
1.1102230246251565e-16
```

Single queries that lie in one clique go through infer as with any backend, with JunctionTreeDiagnostics telling which clique served the query:

```
post, diag = infer(fg, [:bronc, :either]; evidence=ev, backend=JunctionTree())
diag
```

```
JunctionTreeDiagnostics(4, 6, 2, 8, 10, false)
```

```
post ≈ infer(fg, [:bronc, :either]; evidence=ev)[1]
```

```
true
```

A query whose variables are in no common clique falls back to variable elimination (with a warning, and diagnostics.fallback set), so infer(fg, [:asia, :smoke]; backend=JunctionTree()) is still exact and still returns a JunctionTreeDiagnostics. The model-level entry point does the same on a BayesModel:

```
m_ev = observe(asia, :asia => :yes)
round.(all_marginals(m_ev)[:tub].table; digits=4)
```

```
2-element Vector{Float64}:

```

```
 0.05
 0.95
```

5 Belief propagation on a tree

The SPEC section 45 habitat network is a polytree, so its factor graph is a tree and sum-product belief propagation is exact: it converges in a number of sweeps bounded by the diameter, and the residual drops to zero.

```
habitat = compile(reference_habitat_model())
is_tree(habitat)
```

```
true
```

```
bp_marginals, bp_diag = belief_propagation(habitat; evidence=Dict(:Vegetation =>
:dense))
bp_diag
```

```
BPDiagnostics(3, true, 0.0, true, false)
```

```
ve = all_marginals(habitat; evidence=Dict(:Vegetation => :dense),
backend=VariableElimination())
maximum(maximum(abs.(bp_marginals[v].table .- ve[v].table)) for v in
variables(habitat))
```

```
1.1102230246251565e-16
```

6 Loopy belief propagation on asia

Asia has loops (smoke → lung → either → dysp <- bronc <- smoke), so belief propagation is approximate there. It converges, and the marginals are close to the exact ones but not equal:

```
loopy, loopy_diag = belief_propagation(fg)
loopy_diag
```

```
BPDiagnostics(4, true, 1.3877787807814457e-17, false, false)
```

```
exact = all_marginals(fg)
sort([v => (bp=round(loopy[v].table[1]; digits=5), exact=round(exact[v].table[1];
digits=5))
      for v in variables(fg)]; by=first)
```

```

8-element Vector{Pair{Symbol, @NamedTuple{bp::Float64, exact::Float64}}}:
 :asia => (bp = 0.01, exact = 0.01)
 :bronc => (bp = 0.45, exact = 0.45)
 :dysp => (bp = 0.43931, exact = 0.43597)
 :either => (bp = 0.06483, exact = 0.06483)
 :lung => (bp = 0.055, exact = 0.055)
 :smoke => (bp = 0.5, exact = 0.5)
 :tub => (bp = 0.0104, exact = 0.0104)
 :xray => (bp = 0.11029, exact = 0.11029)

```

Damping and the schedule change the path to the fixed point, not the fixed point itself:

```

for backend in (BeliefPropagation(), BeliefPropagation(; damping=0.5),
               BeliefPropagation(; schedule=:sequential))
  ms, d = belief_propagation(fg, backend)
  println((damping=backend.damping, schedule=backend.schedule,
           iterations=d.iterations,
           converged=d.converged, residual=round(d.max_residual; sigdigits=3),
           dysp=round(ms[:dysp].table[1]; digits=6)))
end

```

```

(damping = 0.0, schedule = :flooding, iterations = 4, converged = true, residual =
1.39e-17, dysp = 0.43931)
(damping = 0.5, schedule = :flooding, iterations = 36, converged = true, residual =
8.9e-9, dysp = 0.439311)
(damping = 0.0, schedule = :sequential, iterations = 1, converged = true, residual =
0.0, dysp = 0.43931)

```

Evidence that cuts every loop makes the conditioned graph a tree again, and the diagnostics report `tree = true`, so the fixed point of the message equations is the exact marginal (the field is named `tree` rather than `exact` because the returned `max_residual` measures the undamped message equations at the returned iterate, not a general bound on marginal error):

```

_, cut_diag = belief_propagation(fg; evidence=Dict{:either => :no})
cut_diag

```

```

BPDiagnostics(4, true, 0.0, true, false)

```

VE and JT check global feasibility before returning a posterior, even for an unrelated component or an all-observed query. BP detects some zero-support cases, including the following one, but local messages are not a general global feasibility test.

```

try
  belief_propagation(fg; evidence=Dict{:lung => :yes, :either => :no})
catch e
  typeof(e)
end

```

```

KernelNormalizationError

```

6.1 Slow updates are not convergence

With damping close to one, a tiny update can coexist with a large error. The diagnostic now measures the undamped residual at the returned iterate:

```
coin = FactorGraph([Factor(FiniteAxis(:Coin, [:heads, :tails]), [0.9, 0.1])])
slow, slow_diag = infer(coin, :Coin; backend = BeliefPropagation(damping = 1 -
1e-9))
(belief = slow.table, converged = slow_diag.converged,
residual = slow_diag.max_residual, oracle = infer(coin, :Coin)[1].table)
```

```
(belief = [0.5000000799999865, 0.49999992000001353], converged = false, residual =
0.3999999200000154, oracle = [0.9, 0.1])
```

The iteration cap is reached without convergence; the near-uniform belief is not certified as accurate merely because its damped steps are small.

6.2 Unknown feasibility on a loop

Three pairwise constraints can each have support while being jointly inconsistent. Here $A=B$ and $B=C$ conflict with $A \neq C$:

```
binary(v) = FiniteAxis(v, [:no, :yes])
inconsistent = FactorGraph([Factor([binary(:A), binary(:B)], [1.0 0.0; 0.0 1.0]),
Factor([binary(:B), binary(:C)], [1.0 0.0; 0.0 1.0]),
Factor([binary(:A), binary(:C)], [0.0 1.0; 1.0 0.0])])
_, unchecked = infer(inconsistent, :A; backend = BeliefPropagation())
(converged = unchecked.converged, evidence_checked = unchecked.evidence_checked)
```

```
(converged = true, evidence_checked = false)
```

```
try
    infer(inconsistent, :A; backend = BeliefPropagation(check_evidence = true))
catch e
    typeof(e)
end
```

KernelNormalizationError

The optional exact VE pass detects the contradiction. Its potentially exponential cost is explicit: it is not silently imposed on every loopy-BP run.

7 Timing: all marginals by junction tree versus variable elimination

A random network with 60 binary variables (up to three parents among the previous eight) shows where the junction tree pays off: variable elimination repeats the elimination once per variable, the junction tree calibrates once.

```
ax(v) = FiniteAxis(v, [:a, :b])
function random_network(rng, n; window=8, maxparents=3)
    kernels = Pair{Symbol, FiniteKernel}[]
    parents = Dict{Symbol, Vector{Symbol}}{ }
    for i in 1:n
        v = Symbol(:X, i)
        cands = [Symbol(:X, j) for j in max(1, i - window):(i - 1)]
        ps = shuffle(rng, cands)[1:min(rand(rng, 0:maxparents), length(cands))]
        parents[v] = ps
        dom = FiniteSpace(FiniteAxis[ax(p) for p in ps])
        push!(kernels, v => random_kernel(rng, dom, FiniteSpace(ax(v))))
    end
    return kernels, parents
end

kernels, parents = random_network(MersenneTwister(2026), 60)
big = FactorGraph([Factor(k, parents[v], v) for (v, k) in kernels])
big_ev = Dict{:X60 => :a, :X31 => :b}
(treewidth=build_junction_tree(big).treewidth,
n_cliques=length(build_junction_tree(big)))
```

(treewidth = 4, n_cliques = 48)

```
rows = map((JunctionTree(), VariableElimination(), BeliefPropagation())) do backend
    all_marginals(big; evidence=big_ev, backend) # warm up compilation
    times = [@elapsed all_marginals(big; evidence=big_ev, backend) for _ in 1:3]
    t = minimum(times)
    ms = all_marginals(big; evidence=big_ev, backend)
    reference = all_marginals(big; evidence=big_ev, backend=VariableElimination())
    err = maximum(maximum(abs.(ms[v].table .- reference[v].table)) for v in
variables(big))
    (backend=nameof(typeof(backend)), seconds=round(t; digits=4),
max_error=round(err; sigdigits=3))
end
foreach(println, rows)
(julia = string(VERSION), cpu = Sys.CPU_NAME, threads = Threads.nthreads(),
repetitions = 3, statistic = "minimum after warm-up")
```

(backend = :JunctionTree, seconds = 0.001, max_error = 2.22e-16)
(backend = :VariableElimination, seconds = 0.0235, max_error = 0.0)
(backend = :BeliefPropagation, seconds = 0.0109, max_error = 0.0521)

(julia = "1.12.7", cpu = "apple-m1", threads = 1, repetitions = 3, statistic =
"minimum after warm-up")

These timings describe this run, not a portable performance guarantee.

The junction tree gives the exact answers of variable elimination at a fraction of its cost (one calibration instead of one elimination per variable), and the gap widens with the number of variables. Loopy belief propagation is cheap too, but on this loopy graph its marginals are off by several percent; it is the backend for graphs too dense to triangulate, not a substitute for the junction tree when the treewidth is small.

8 Summary

A junction tree turns the factor graph into a tree of cliques and calibrates it with one collect-and-distribute sweep of Shafer-Shenoy messages (Shafer and Shenoy 1990), so every single-variable marginal comes out of a single pass instead of one elimination per variable; because the messages are never divided by a separator potential, deterministic tables with zero entries need no special treatment, which is the practical difference from the division-based architectures of Lauritzen and Spiegelhalter (1988) and Jensen et al. (1990). Sum-product belief propagation (Pearl 1988; Kschischang et al. 2001) on the factor graph itself is exact on a tree and approximate with cycles, where the loopy fixed point is the empirical object studied by Murphy et al. (1999). The next vignette, *Validation and scoring*, asks the separate question of whether the answers a calibrated network gives are any good.

9 References

- Jensen, Finn V., Steffen L. Lauritzen, and Kristian G. Olesen. 1990. “Bayesian Updating in Causal Probabilistic Networks by Local Computations.” *Computational Statistics Quarterly* 4: 269–82.
- Kschischang, Frank R., Brendan J. Frey, and Hans-Andrea Loeliger. 2001. “Factor Graphs and the Sum-Product Algorithm.” *IEEE Transactions on Information Theory* 47 (2): 498–519. <https://doi.org/10.1109/18.910572>.
- Lauritzen, Steffen L., and David J. Spiegelhalter. 1988. “Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems.” *Journal of the Royal Statistical Society, Series B* 50 (2): 157–224. <https://doi.org/10.1111/j.2517-6161.1988.tb01721.x>.
- Murphy, Kevin P., Yair Weiss, and Michael I. Jordan. 1999. “Loopy Belief Propagation for Approximate Inference: An Empirical Study.” *Proceedings of the Fifteenth Conference on Uncertainty in Artificial Intelligence (UAI 1999)*, 467–75.
- Pearl, Judea. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.

Shafer, Glenn R., and Prakash P. Shenoy. 1990. "Probability Propagation." *Annals of Mathematics and Artificial Intelligence* 2: 327–51. <https://doi.org/10.1007/BF01531015>.